

VISIONINSPECT-AI

Manufacturing Defect Detection & Quality Inspection System

Chamarthy Lakshyatha
Backend Development

Table of Contents

1. ANALYTICS AND REPORT EXPORT	3
2. APPLICATION INTEGRATION – app.py	3
3. DATABASE PATH MANAGEMENT	3
4. DYNAMIC FILE CONFIGURATION	4
5. INSPECTION AND HEATMAP INTEGRATION	4
6. SUPERVISOR REPORT EXPORT TEST	4
7. COMPLETE UPDATED BACKEND FLOW	5
8. COMPLETED BACKEND IMPROVEMENTS	5
9. FINAL STATUS	5

1. ANALYTICS AND REPORT EXPORT

The analytics module has been enhanced to support report export functionality.

API: GET /api/analytics/export

- Export analytics reports in JSON and CSV formats.
- Support filtering of analytics data.
- Provide downloadable CSV reports.
- Use persisted inspection information rather than rerunning AI inference.

Report Route: GET /api/reports/export

The report export route is registered through app.py and integrated with the analytics functionality.

Export Flow

```
Frontend
↓
Report Request
↓
Authentication
↓
Authorization
↓
Analytics Export
↓
Database Query
↓
Filtering / Aggregation
↓
JSON / CSV
↓
Response / Download
```

2. APPLICATION INTEGRATION – app.py

- Registers active analytics routes.
- Registers the report-export route.
- Maintains centralized configuration and database initialization.
- Provides the active Flask application integration layer.

Updated Flow

app.py → Analytics Routes → Report Export → JSON / CSV Response

3. DATABASE PATH MANAGEMENT

A centralized database path helper has been introduced in database_utils.py.

- get_db_path() resolves the database location.
- The database directory is created automatically when required.
- get_db_connection() now uses the centralized database path.
- connection.py uses database_utils.get_db_path().
- create_supervisor.py no longer uses a hardcoded database path.

This improves portability, reduces duplicated configuration, and keeps database access consistent.

4. DYNAMIC FILE CONFIGURATION

The upload and inspection modules now obtain allowed file extensions dynamically from the active Flask application configuration.

upload.py

`allowed_file()` reads `ALLOWED_EXTENSIONS` from application configuration at runtime.

inspection.py

Direct configuration imports were removed. Configuration is accessed through the Flask application, and `allowed_file()` uses the application configuration.

File Flow

Upload Request → `allowed_file()` → `ALLOWED_EXTENSIONS` → Extension Validation → Secure Filename → Storage

5. INSPECTION AND HEATMAP INTEGRATION

Inspection: AI defect-detection results and quality-control decisions are persisted correctly so they remain available for history, analytics, reporting, and dashboard visualization.

Heatmap: `heatmap.py` now reads `HEATMAP_FOLDER` from application configuration at runtime and uses the corrected output path.

Updated Inspection Flow

Frontend → `/api/inspection/predict` → Authentication → Validation → Image Processing → AI Inference → Defect Detection → Heatmap → Severity → Quality Decision → SQLite Persistence

6. SUPERVISOR REPORT EXPORT TEST

A new automated test case has been added:

`test_supervisor_can_export_reports`

- Verifies that an authenticated Factory Supervisor can export reports.
- Validates report-export authorization.
- Verifies JSON/CSV report-export functionality.

Testing Status: Backend functionality was verified after the updates, including report export, configuration handling, database path management, heatmap handling, and inspection persistence.

7. COMPLETE UPDATED BACKEND FLOW

The updated implementation integrates the new functionality into the existing backend workflow:

```
FRONTEND
↓
Authentication & JWT Authorization
↓
REST API Layer
↓
File Upload / Inspection
↓
Dynamic Application Configuration
↓
Image Validation
↓
AI Inference
↓
Heatmap Generation
↓
Severity Scoring
↓
Quality Decision
↓
SQLite Persistence
↓
History / Analytics
↓
JSON / CSV Report Export
↓
Frontend Dashboard
```

8. COMPLETED BACKEND IMPROVEMENTS

Area	Completed Update
Analytics & Reporting	JSON/CSV export, filtering, CSV download, /api/reports/export integration
Configuration	Centralized database path; dynamic ALLOWED_EXTENSIONS and HEATMAP_FOLDER
Database	get_db_path(), automatic directory creation, centralized connections
Inspection	Application-level configuration and improved result/quality-decision persistence
Heatmap	Runtime folder configuration and corrected output path
Testing	Supervisor report-export test and post-update backend verification

9. FINAL STATUS

The backend updates have been integrated into the existing VisionInspect-AI architecture. The changes improve centralized configuration, database portability, reporting, inspection-result persistence, heatmap handling, and automated test coverage while preserving the modular Flask architecture.